

포트폴리오 | 김선평

실무 사례 3건을 다룹니다. 세 사례 모두 본인이 설계·구현·검증을 단독으로 담당했습니다. 전·현직 회사의 영업비밀을 보호하기 위해 회사 고유의 서비스명, 내부 호스트명, IP, 금액, 보안 규칙의 예외 경로, 소스 코드는 실지 않았습니다. 공개된 오픈소스와 AWS 기능을 어떻게 판단하고 적용했는지만 일반화해 기술했습니다.

- 1. GitOps 플랫폼 전환과 소유 경계 설계 (DXE: 플랫폼을 내부 제품처럼)
- 2. 관측 체계 재구축과 서비스 DOWN SLI (SRE: Observability를 기본값으로)
- 3. 포스트모템: 기동 실패한 웹앱이 만든 DB 커넥션 누수 (SRE: 같은 장애를 두 번 겪지 않기)

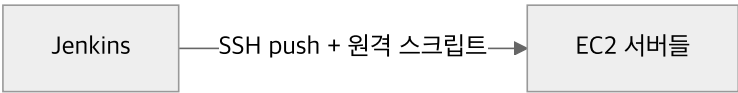
Case 1. GitOps 플랫폼 전환과 소유 경계 설계

배경

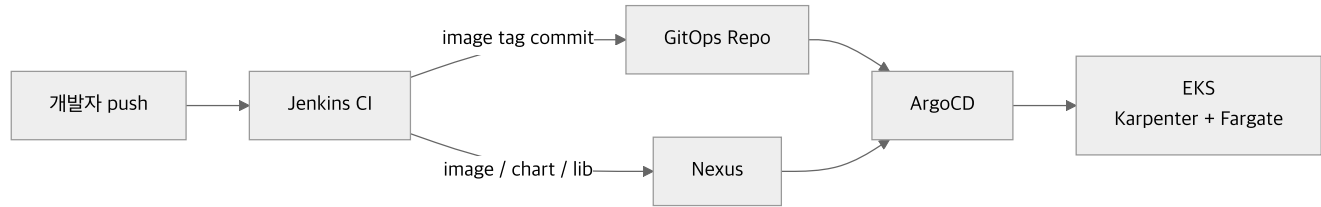
레거시는 Jenkins가 빌드 산출물을 SSH로 서버에 밀어 넣고 원격 스크립트를 실행하는 구조였습니다. 서비스마다 스크립트가 달랐고, 어떤 버전이 어디에 배포됐는지는 사람이 기억해야 했습니다.

Before → After

Before



After



설계 원칙

원칙	내용
역할 분리	Nexus = 아티팩트, Git = 배포 결정의 상태. Jenkins는 commit까지만, 배포는 ArgoCD 전담
조합 가능한 파이프라인	Shared Library = 원자 함수 7개(서로 참조 안 함) + 조합 템플릿 4개
구조로 사고 방지	운영은 Build/Deploy Job 분리. Deploy는 Nexus의 최신 태그를 스스로 조회하고 수동 승인을 거침. 운영 레지스트리에는 삭제 권한 없음
Self-service	App-of-Apps. 신규 서비스는 Application.yaml + values.yaml 2개 파일만 추가
소유 경계	"쿠버네티스 API 밖이면 Terraform, 안이면 ArgoCD. 예외는 부트스트랩뿐"

소유 경계 재설계

리소스	AS-IS	TO-BE
VPC·EKS·IAM·IRSA	Terraform	Terraform
ArgoCD, Karpenter (부트스트랩)	Terraform	Terraform (예외)
ESO·Traefik·Gateway API CRD·ALB Controller 차트	Terraform	ArgoCD (sync wave -2 → 0 → 1)
Ingress / 내부 NLB Service	수동 <code>kubectl apply</code> (주인 없음)	ArgoCD
ArgoCD 계정·RBAC	Terraform patch	ArgoCD selfHeal (부분 소유, Server-Side Apply)

효과

- 상류 모듈이 바뀌면 `plan` 이 깨지던 `Invalid for_each` 문제가 구조적으로 사라졌습니다.
- CRD의 `last-applied` 크기가 248KB(한계의 95%)에서 0이 되었습니다.
- 검증: `terraform plan` No changes, Application 16개 Synced/Healthy

제어면을 Fargate로 옮길 때 순서를 뒤집은 이유

- **원래 계획:** requests를 먼저 조정한 뒤 Fargate로 이동
- **계산 결과:** 노드 CPU requests가 이미 85~90%였습니다. requests를 먼저 올리면 순증분이 기존 노드에 얹혀 Karpenter가 3번째 노드를 띄웁니다.
- **실행:** Fargate 프로파일을 먼저 만들어, 새 파드가 처음부터 Fargate에 뜨게 했습니다. 노드는 2대로 유지됐고, **앱 파드 재시작 0건, UI 무중단**이었습니다.
- **트레이드오프 명시:** Fargate는 OS 패치 시점을 AWS가 정하고 콜드스타트도 느립니다. 그래서 전환 근거는 '안정성'이 아니라 '제어면과 데이터플레인의 분리'라고 문서에 적었습니다.

운영 중 발견한 문제와 구조적 해결

문제	근본 원인	해결
이미지 태그 덮어쓰기	Job을 재생성하면 빌드 번호가 1로 리셋	<code>env-날짜-빌드번호-SHA8</code> 형식 + OCI 라벨
Nexus 디스크 100%	'최근 N개 보존' 기능 미지원 + 업로드 스테이징 누수	4단 삭제 체인 규명, 파이프라인 끝단에 보존 자동화(안전장치 6중, 실패해도 배포는 유지). 디스크 사용률 70% → 29%
동시 빌드 실패	Gradle 캐시 공유로 lock 경합, GitOps push 충돌	서비스별 캐시 격리, fetch 후 재적용 재시도(최대 3회, 이후 실패 처리)
Helm upgrade 후 ArgoCD 권한 소실	차트 기본값 <code>policy.csv: ""</code> 가 덮어씀 (admin은 RBAC를 우회해 드러나지 않음)	GitOps 이관 + 일부러 망가뜨려 25초 내 자동 원복을 실증 + AppProject로 권한 분리

Case 2. 관측 체계 재구축과 서비스 DOWN SLI

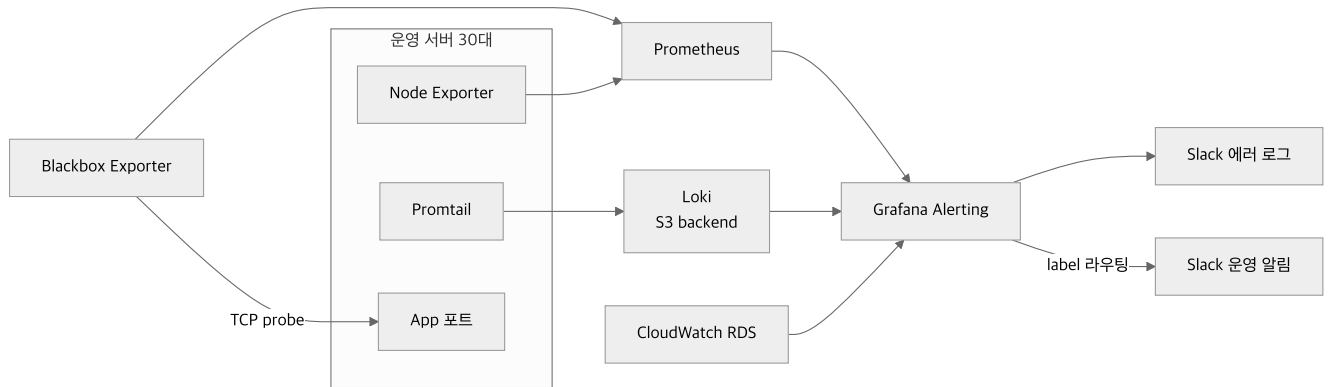
배경

- 레거시 관제 시스템(Logstash + JDBC + 자체 웹앱)을 대체하려고 상용 APM을 PoC했습니다.
- 그런데 에이전트 기능을 켜자 운영 인스턴스 6대에서 OOM Kill이 발생했습니다.
- 소형 인스턴스가 많은 환경에서는 에이전트 자체가 장애 원인이 될 수 있다고 판단해 철수를 결정했습니다.

선택 근거 (실측)

후보	메모리	판단
Node Exporter + Promtail	약 80MB	채택 (관리 포인트는 늘지만 장애 리스크가 낮음)
Grafana Alloy	235MB 이상	제외

아키텍처



핵심 결정

- **전환 근거를 먼저 만들었습니다:** 레거시 관제의 이벤트 13종, 임계치, 알림 채널, 발송 시간대를 역공학해 Grafana 이관 매핑표를 만든 뒤 전환했습니다.
- **서비스 DOWN SLI:** '수집이 끊기면 알림'이 아니라 앱 포트 40여 개에 대한 직접 TCP probe(`probe_success == 0`)로 정의했습니다. `instance-service` 라벨로 어느 서버의 어느 앱인지 바로 식별됩니다.
- **대상 목록은 3중 교차검증:** 포트 매핑을 `ss -tlnp`, 프로세스 작업 디렉토리, 관리 도구, 인프라 문서로 대조했습니다.
- **알림 피로도 관리**
 - 복구 알림 필요 여부에 따라 Contact Point를 분리했습니다.
 - 에러 로그 알림은 10분간 반복을 억제합니다.
 - 기존 에러가 상시 발생하는 서버는 제외했습니다.
- **알림은 바로 행동할 수 있게:** 메시지에 인스턴스명, 현재 값, 해당 인스턴스 대시보드 링크를 넣었습니다.
- **배포:** SSM으로 30대에 일괄 배포했습니다. `positions offset`을 먼저 기록해 과거 로그가 중복 수집되지 않게 했습니다.
- **표준화:** WARN 알림은 SSM으로 테스트 로그 주입 → Loki 수집 → firing → Slack 수신까지 E2E로 검증한 뒤 표준으로 정리했습니다.

Case 3. 포스트모템: 기동 실패한 웹앱이 만든 DB 커넥션 누수

요약

항목	내용
증상	QA DB 커넥션 풀이 주기적으로 15 → 84개 급증
영향	QA DB 커넥션 포화 위험 (운영은 영향 없음, 단 '우연히')
근본 원인	기동에 실패한 웹앱의 DBCP Evictor 스레드가 살아남아 30초마다 커넥션을 생성하고 고아화
결과	좀비 스레드 1 → 0, 커넥션 82 → 1(호스트 전체 109 → 28), 누수 로그 하루 960건 → 0

원인 분석

1. 웹앱 컨텍스트 초기화가 실패했지만, 실패 직전에 생성된 DataSource의 Evictor(Timer) 스레드는 살아남았습니다.
2. 이 스레드는 설정(30초)대로 커넥션을 열었습니다. TCP 연결은 성공했지만, 이미 정지된 클래스로더 때문에 다음 단계에서 예외가 났고 소켓이 열린 채 고아가 됐습니다.
3. Tomcat 기본값(`clearReferencesStopThreads=false`)은 경고만 남기고 스레드를 죽이지 않습니다. 그래서 파일 삭제만으로는 멈추지 않고, **JVM 재기동이 필요**했습니다.

"운영이 안전했던 것은 우연"

환경	기동 실패 지점	DataSource 생성	누수
QA (Tomcat 7)	구버전 JDBC 드라이버가 Aurora 8.0 핸드셰이크 파싱 실패	생성됨	발생
운영 (Tomcat 8.5)	DBCP 패키지명 변경으로 더 이른 단계에서 ClassNotFound	생성 안 됨	없음

클래스 오류 하나만 고치면 운영에서도 같은 누수가 재현될 상태였습니다. 이 웹앱은 약 5년 동안 기동 실패 상태로 아무 서비스도 하지 않고 있었습니다. 이 점을 근거로 제거를 결정했습니다.

안전한 제거 절차

- `rm` 대신 `mv` 로 appBase 밖으로 이동해 즉시 복원할 수 있게 했습니다.
- autoDeploy가 다시 전개하지 않도록 war를 먼저, 디렉토리를 나중에 옮겼습니다. 롤백은 역순입니다.
- 백업은 압축본과 실물로 이중 보관하고, 엔트리 수와 gzip 무결성을 대조했습니다.
- 운영 2대는 서비스 재기동 없이 처리했습니다(배포 파일 해시와 리스닝 포트 유지 확인). QA는 해당 프로세스만 재기동했습니다.

남긴 것

- 환경별로 실패 지점이 다를 수 있다는 점을 포함한 원인 분석 문서
- 잔존 여부를 검사할 범위 목록(webapps, work 디렉토리, 자동 배포 경로, 배포 스크립트)
- 레거시 웹앱 제거 절차(이동 → 검증 → 롤백 순서)